NEERAJ SINGH
DEVOPS

1. What is RAG ?

Retrieval-Augmented Generation (RAG) is a method that combines information retrieval with a language model. Before answering, the system searches a trusted knowledge source and gives relevant information to the model as context.

Key Points :

- Retrieves information at query time.
- Grounds answers in selected source content.
- Can use private or frequently updated documents.
- Helps reduce unsupported answers but does not remove errors.
- Quality depends on retrieval, context and generation.

Simple Example :

- Question : What is our leave policy ?
- Retrieval : Find relevant HR policy paragraphs.
- Generation : Answer using those paragraphs.

2. How RAG Works

A RAG pipeline retrieves relevant passages for a user query and supplies them to a language model to generate a grounded response.

General Process :

1. Collect documents.
2. Clean and split text into chunks.
3. Create embeddings for each chunk.
4. Store vectors and metadata in an index.
5. Embed the user query and retrieve relevant chunks.
6. Build a context and generate an answer.
7. Check the answer and show sources when available.

Simple Flow :



Important Concepts :

- Retrieval selects evidence; generation writes the answer.
- Chunking and search quality affect relevance.
- RAG can cite sources, but citations must be verified.

3. Why RAG is Needed ?

Large Language Models are trained on a fixed set of data. They may not have the latest information and can sometimes generate incorrect answers (hallucinations). RAG helps by providing relevant and up to date information from external sources.

Key Advantages :

- Provides up to date and domain specific information.
- Reduces hallucinations.
- Can use private and internal documents.
- More reliable and factual answers.
- Useful for real world business applications.

Example :

Without RAG, the model may not know your company's latest policy. With RAG, it can retrieve the policy from your internal document and answer accurately.

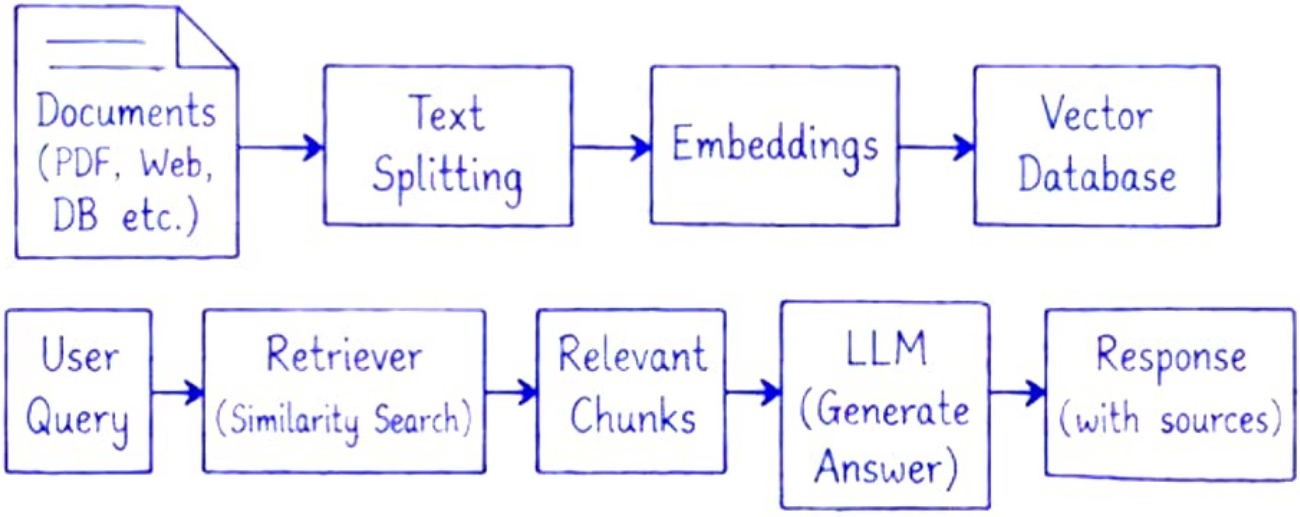
LLM alone (limited knowledge)	RAG (up to date and (custom knowledge)

4. Key Components of RAG

A typical RAG system has the following components:

1. Data Sources - Documents, websites, databases etc.
2. Document Loading - Read and prepare the data.
3. Text Splitting - Split documents into smaller chunks.
4. Embeddings - Convert text into vector representations.
5. Vector Database - Store and search embeddings.
6. Retriever - Find relevant chunks for a query.
7. LLM - Generate the final answer using retrieved context.
8. Response - Return answer with source (optional).

Architecture Diagram :



5. Data Sources for RAG

RAG can use different types of data sources depending on the use case. The data can be public, private or real time.

Common Data Sources :

- PDF documents
- Word and Excel files
- Internal company documents
- Web pages and websites
- Databases (e.g. MySQL, PostgreSQL)
- APIs and online knowledge bases
- Notion, Confluence and other tools

Points to Consider :

- Data should be relevant and high quality.
- Remove unnecessary or duplicate content.
- Handle different file formats (PDF, DOCX, TXT etc.).
- Keep data updated regularly.
- Follow security and access control for private data.
- Large documents should be split into smaller chunks.

6. Document Loading and Parsing

Document loading is the first step in a RAG pipeliee. It involves reading data from different sources and converting it into a format that can be processed.

Common Document Sources :

- PDF files
- Word and Excel files
- Text files
- Web pages (URLs)
- Databases (e.g. MySQL, PostgreSQL)
- Cloud storage (e.g. Google Drive, S3)
- APIs and online sources

Parsing and Extraction :

- Extract text from documents.
- Remove headers, footers, page numbers and noise.
- Handle tables, images and scanned documents (using OCR when needed).
- Convert content into clean plain text.

Popular Tools and Libraries :

- LangChain document loaders
- PyPDF2, pdfplumber for PDF
- python-docx for Word files
- pandas for Excel files
- BeautifulSoup for web pages
- Tesseract OCR for scanned PDFs

7. Text Splitting (Chunking)

After extracting text, the content is split into smaller chunks. Chunking helps in better retrieval because smaller pieces of text are easier to search and more relevant to the user query.

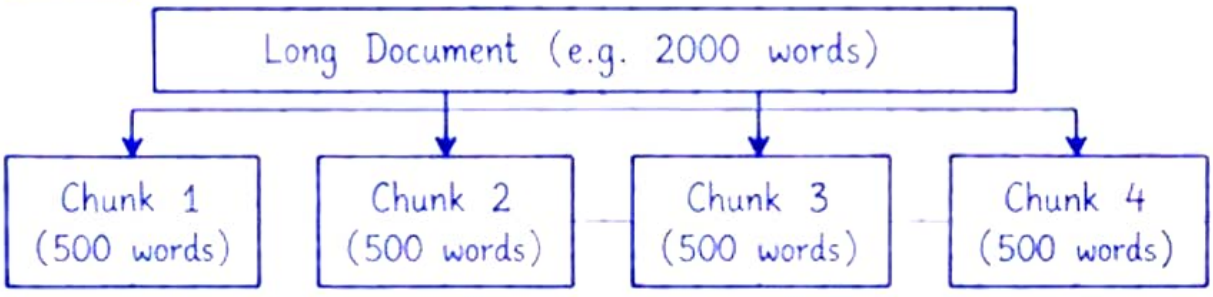
Key Points :

- Chunks should be meaningful and complete.
- Avoid very small or very large chunks.
- Maintain context while splitting.
- Use overlap between chunks to avoid missing information.
- Chunk size depends on the data and use case.

Common Chunking Strategies :

- Fixed size chunking (e.g. 500 characters)
- Recursive character splitting (based on text structure)
- Semantic chunking (based on meaning)
- Sentence based chunking
- Paragraph based chunking

Example :



Overlap Example :

- Chunk size = 500 characters
- Overlap = 50 characters (to maintain context)

8. Embeddings and Vector Databases

Embeddings convert text into numerical vectors that capture the meaning of the text. Similar texts have similar vectors. These vectors are stored in a vector database for fast similarity search.

What are Embeddings ?

- Text is converted into high dimensional vectors.
- Vectors capture the semantic meaning.
- Used to find similar content based on meaning, not just exact keywords.

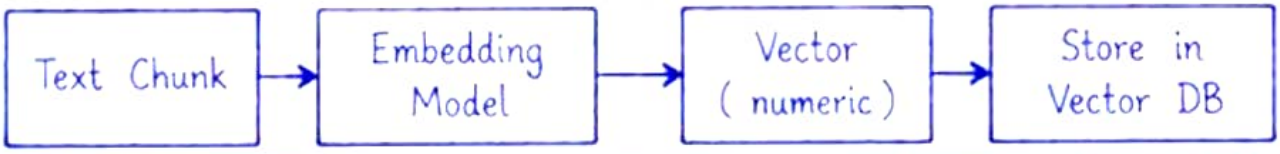
Popular Embedding Models :

- OpenAI Embeddings (text-embedding-3-small, text-embedding-3-large)
- Sentence Transformers (e.g. all-MiniLM-L6-v2)
- Cohere Embeddings
- Hugging Face models

Vector Databases :

- FAISS - open source, good for small to medium data
- Pinecone - managed cloud service
- Weaviate - open source, scalable
- Chroma - easy to use, good for local development
- Milvus - high performance, scalable
- Qdrant - open source vector database

How it Works :



9. Retriever in RAG

The retriever is responsible for finding the most relevant chunks from the vector database based on the user query. It converts the query into an embedding and searches for similar embeddings.

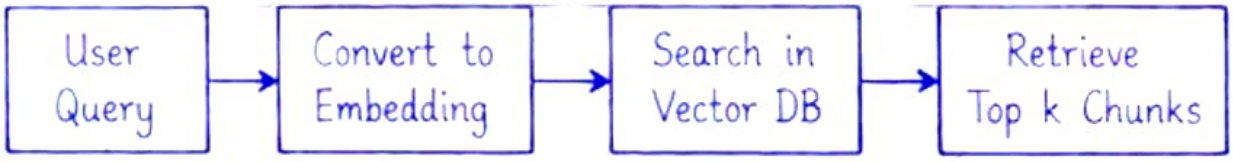
Types of Retrievers :

- Similarity Search - Find top k most similar chunks.
- Keyword Search - Use keywords (BM25).
- Hybrid Search - Combine keyword search and vector search.
- Reranker - Reorder the retrieved chunks for better relevance.

Key Points :

- Retriever should return relevant and diverse chunks.
- Top k value affects the quality of answer.
- Similarity metrics like cosine similarity are commonly used.
- Hybrid retrieval gives better results in many cases.

Simple Retriever Flow :



Common Libraries :

- LangChain - vector store retrievers, hybrid search.
- LlamaIndex - different retrieval strategies.
- Custom retrievers - for specific use cases.

10. Reranking and Relevance Filtering

The initial retrieval may return multiple chunks, but not all of them are equally relevant. Reranking and filtering help in selecting the most useful information before passing it to the LLM.

Why Reranking ?

- Initial retrieval is based on similarity, which may not always capture exact relevance.
- Reranking uses a more advanced model to reorder chunks.
- Improves the quality and accuracy of the final answer.

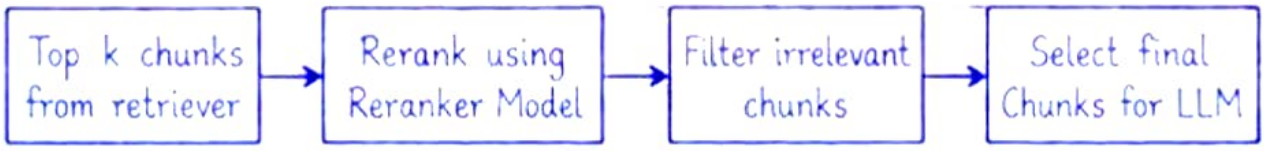
Common Reranking Methods :

- Cross Encoder models (e.g. bge-reranker, Cohere rerank)
- LLM based reranking.
- Rule based filtering (remove irrelevant or low quality chunks).
- Metadata based filtering (e.g. date, source, category).

Key Points :

- Reranking improves precision of retrieved information.
- Use top k (e.g. 5 to 10) chunks after reranking.
- Remove duplicate or very similar chunks.
- Consider chunk diversity to cover different aspects.

Example Flow :



11. Prompting in RAG

Prompting is very important in RAG because it helps the LLM use the retrieved context correctly to generate a good and accurate answer. A well designed prompt reduces hallucinations and ensures the model sticks to the provided information.

Key Elements of a RAG Prompt :

- System instruction - tells the model how to behave.
- User query - the actual question.
- Retrieved context - relevant chunks from the database.
- Output format - optional (e.g. bullet points, table, summary).

Example Prompt Template :

```
You are a helpful assistant. Use the following context
to answer the question. If the answer is not in the
context, say that you do not know.
Context:
{retrieved_chunks}

Question: {user_query}

Answer:
```

Best Practices :

- Keep the prompt clear and concise.
- Include only relevant context.
- Specify the desired output format.
- Ask the model to avoid making up information.
- Customize the prompt for your specific use case.

12. LLM in RAG

In RAG, the LLM is used to generate the final answer using the retrieved context. The LLM does not need to know everything, it only needs to use the provided information to give a useful, accurate and grounded response.

Role of LLM in RAG :

- Understands the user query.
- Uses retrieved context to generate answer.
- Follows instructions from the prompt.
- Can summarize, explain, compare and analyze information.
- Reduces hallucinations by relying on given context.

Key Points :

- LLM should be provided with clear and relevant context.
- Use instructions to make the answer grounded.
- Long context may affect performance, so provide only relevant chunks.
- Output style can be controlled using prompt.

Simple Flow :



13. RAG Architectures

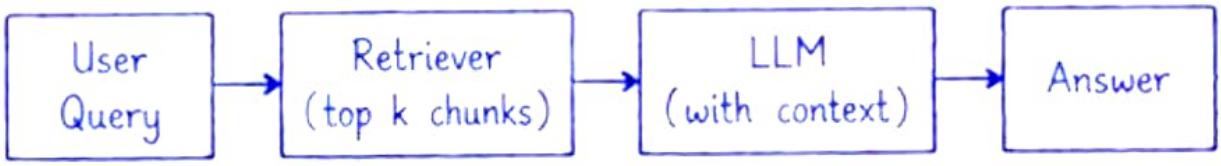
There are different types of RAG architectures based on how retrieval and generation are combined. The choice depends on the use case, data size and performance needs.

Common RAG Architectures :

1. Naive RAG - Simple retrieve and generate.
2. Advanced RAG - Uses reranking, query rewriting and other techniques.
3. Modular RAG - Different modules for retrieval, reranking and generation.
4. Agentic RAG - Uses an agent to decide next steps like search, tools or multiple queries.
5. Graph RAG - Uses knowledge graphs for better context and relationships.
6. Hybrid RAG - Combines vector search, keyword search and other methods.

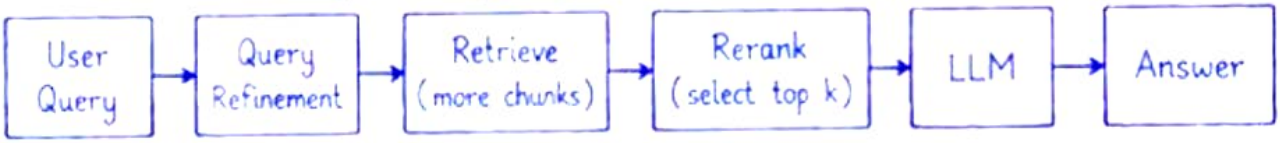
1. Naive RAG Architecture :

Simple and commonly used for basic applications.



2. Advanced RAG Architecture :

Includes reranking and query refinement.



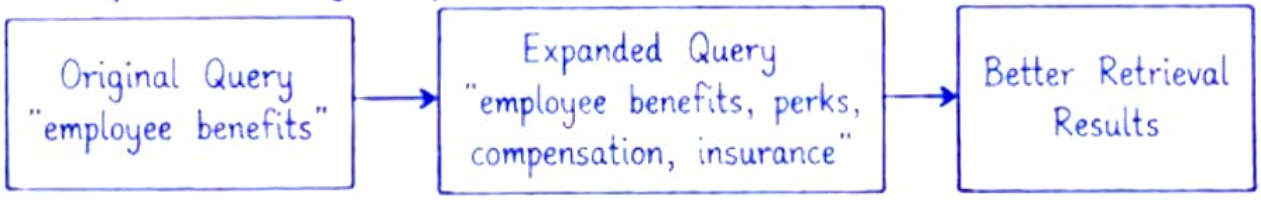
14. Advanced RAG Techniques

To improve the accuracy and reliability of RAG, several advanced techniques are used. These techniques help in better retrieval, better context understanding and more grounded answers.

Key Techniques :

1. Query Expansion - Add related terms to the query.
2. Query Rewriting - Rephrase the query for better search.
3. Hybrid Retrieval - Combine vector and keyword search.
4. Reranking - Reorder retrieved chunks for relevance.
5. Multi Query Retrieval - Generate multiple queries and combine results.
6. Metadata Filtering - Filter results based on source, date or category.
7. Context Compression - Reduce chunk size to keep only important information.
8. Self RAG - Model decides when to retrieve and when to answer directly.
9. Agentic RAG - Use an agent to plan multiple steps like search, tool use and verification.
10. Graph RAG - Use knowledge graphs for structured and relational data.

Example : Query Expansion



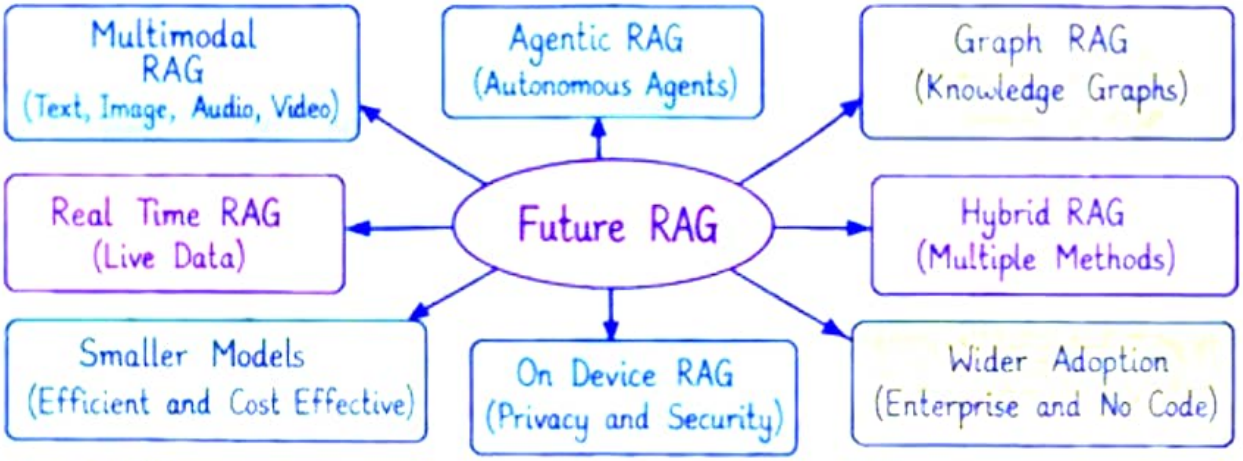
49. Future Trends in RAG

RAG is a rapidly evolving field with many exciting advancements in the coming years. New models, better retrieval techniques and integration with multi modal and agentic systems will make RAG more powerful, accurate and widely adopted across industries.

Key Future Trends :

1. Multimodal RAG - working with text, images, audio and video.
2. Agentic RAG - autonomous agents planning and using multiple tools.
3. Graph RAG - better reasoning with structured knowledge.
4. Real Time RAG - using live data from web, APIs and databases.
5. Hybrid RAG - combining multiple retrieval methods.
6. Smaller and Efficient Models - high performance with lower cost.
7. On Device RAG - running RAG on local devices for privacy.
8. Domain Specific RAG - specialized systems for healthcare, legal, finance, education etc.
9. Improved Evaluation - standardized benchmarks and metrics.
10. Wider Adoption - enterprise tools and no code platforms.

Future RAG Ecosystem :



Key Takeaways :

- RAG will become more multimodal and agent driven.
- Real time and graph based retrieval will improve accuracy.
- It will be more accessible with no code tools.
- RAG will play a key role in enterprise and domain specific applications.

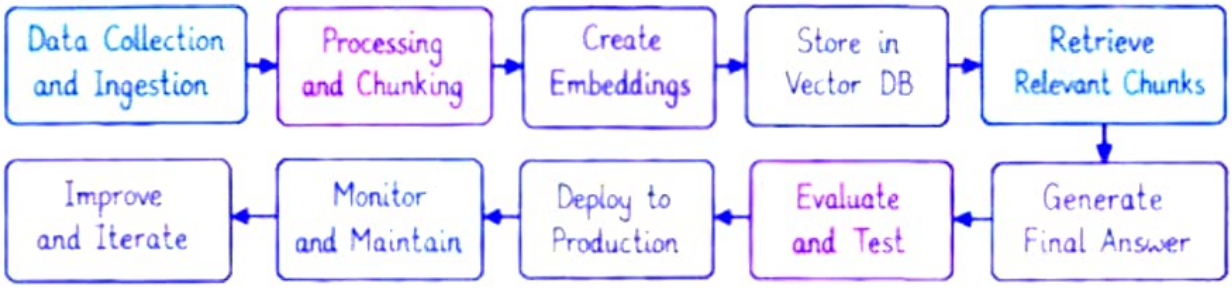
50. Conclusion and Key Takeaways

RAG combines the power of retrieval with the reasoning capabilities of Large Language Models. It helps in generating accurate, reliable and up to date answers by using external knowledge sources. Throughout this handbook, we have covered the complete RAG journey from basics to advanced topics, including implementation, evaluation, deployment and real world applications.

Key Takeaways :

1. RAG is useful for real world applications where accurate and up to date information is required.
2. Good date preparation and chunking are essential for better retrieval.
3. The choice of embeddings, vector database and retrieval strategy significantly impacts performance.
4. Advanced techniques like hybrid search, reranking, graph RAG and agentic RAG can handle complex use cases.
5. Evaluation, monitoring and security are important for production systems.
6. RAG is evolving rapidly with multimodal capabilities, real time data and agentic workflows.
7. With the right tools and best practices, anyone can build powerful RAG applications.

Complete RAG Lifecycle :



Final Words :

RAG is not just a technique, it is a bridge between the vast knowledge in the world and the reasoning power of LLMs. As the technology continues to evolve, it will unlock new possibilities across industries and create more intelligent, reliable and useful AI applications.

classmate

Date

Page

Table of Contents

Page No.	Topic
1	Cover Page
2	Table of Contents
3	Introduction to RAG
4	How RAG Works
5	RAG vs Fine Tuning
6	Components of RAG
7	Data Preparation for RAG
8	Document Loading
9	Text Chunking
10	Embeddings in RAG
11	Vector Databases
12	Retrieval Methods
13	Similarity Search
14	Hybrid Search in RAG
15	Reranking in RAG
16	Prompt Engineering for RAG
17	Context Compression
18	Advanced Prompting Techniques
19	RAG Architectures
20	RAG Evaluation Metrics
21	Query Transformation in RAG
22	Context Compression in RAG
23	Self RAG
24	Agentic RAG
25	Graph RAG
26	Knowledge Graphs in RAG
27	Corrective RAG (CRAG)
28	Multimodal RAG
29	RAG for Structured Data
30	RAG with APIs and External Tools
31	RAG Pipeline End to End
32	Building a RAG Application (Part 1)
33	Building a RAG Application (Part 2)
34	Real World Applications of RAG
35	Testing and Evaluation of RAG
36	Common Issues and Troubleshooting
37	Security and Privacy in RAG
38	Deployment of RAG Applications
39	Monitoring and Maintenance in Production
40	Advanced RAG Techniques and Variations
41	RAG with Agentic Workflows
42	RAG with Knowledge Graphs
43	RAG Interview Questions
44	Building a Complete RAG Project
45	Code Implementation for RAG Project
46	Best Practices for Building RAG Applications
47	RAG Cheat Sheet and Quick Revision Guide
48	Glossary of Important Terms
49	Future Trends in RAG
50	Conclusion and Key Takeaways